

# LAB 4a

---

Krishna Jaiswal 2025CS11180

Ankita 2025CS11180

## 1. Functional Specification

| Signal           | Width  | Function                                  |
|------------------|--------|-------------------------------------------|
| A                | 8 bits | First input word                          |
| B                | 8 bits | Second input word                         |
| threshold        | 4 bits | Maximum allowed Hamming distance          |
| difference       | 8 bits | A XOR B; shows which bit positions differ |
| distance         | 4 bits | Number of 1s in difference; range 0–8     |
| equal            | 1 bit  | 1 exactly when A and B are identical      |
| within_threshold | 1 bit  | 1 exactly when distance $\leq$ threshold  |

## 2. Design Overview

The top module **hamming\_distance8** connects four main functions:

1. XOR generation of the 8-bit difference vector.
2. Counting of bits set to one
3. 8-bit comparison to check if values are equal.
4. A 4-bit comparison to compare two values.

Bit count is created using adders instead of one single arithmetic operation.

## 3. Module Description

The design is implemented using the following modules:

### 2.1 full\_adder

Implements a bit full adder, use arithmetic building block in my design.

### 2.2 popcount8

The popcount8 module counts the number of 1s in an 8-bit input. Calculate the number of differing bits between the two input vectors.

### 2.3 comparator4

Determines whether the first value is less than or equal to the second value.

## 2.4 comparator8

Determines whether they are equal

## 2.5 hamming\_distance8

Produces:

- `difference`
- `distance`
- `equal`
- `within_threshold`

Calculate and compare the Hamming distance between `A` and `B`.

## 4. Important Design Properties

| Property                  | Why it holds                                                                           |
|---------------------------|----------------------------------------------------------------------------------------|
| $d(A,A) = 0$              | $A \text{ XOR } A$ is all zeros, so the population count is zero.                      |
| $d(A,B) = d(B,A)$         | XOR is commutative, so $A \text{ XOR } B = B \text{ XOR } A$ .                         |
| $0 \leq d(A,B) \leq 8$    | There are only eight bit positions, so at most eight positions can differ.             |
| $\text{threshold} \geq 9$ | The distance can be at most 8, so these threshold values always satisfy the condition. |

## 5. Testbench and Verification

The self-checking testbench applies input combinations to the design and calculates the expected Hamming distance independently using a behavioral reference function. The result from the design is then compared with the reference result.

The testbench checks:

- The `difference` vector.
- The `distance` output.
- The `equal` output.
- The `within_threshold` output.
- All 256 values of `A`.
- All 256 values of `B`.
- Threshold values from 0 to 15.

If all threshold values are tested for every possible `A/B` pair, the total number of test vectors is:

$$256 \times 256 \times 16 = 1,048,576$$

A mismatch counter records any incorrect output, and the simulation finishes with a clear `PASS/FAIL` result instead of relying only on visual waveform inspection.

## 6. Example Test Case

For:

```
A      = 10110100
B      = 10011100
threshold = 3
```

the expected outputs are:

| Output           | Expected value |
|------------------|----------------|
| difference       | 00101000       |
| distance         | 2              |
| equal            | 0              |
| within_threshold | 1              |

The XOR result contains two 1s, so the Hamming distance is 2.

Since the two input words are different, `equal = 0`.

Since  $2 \leq 3$ , `within_threshold = 1`.

## 7. Conclusion

The circuit creates XOR difference vector adds up the bits using full adders checks if the 8 bits are the same and looks at the 4-bit distance compared to a limit.

The design is split into parts, which helps make the way it works easier to see and fix problems. The self-checking testbench covers all A/B input pairs and every threshold value that the testbench uses.